Eraser.io Prompt Collection

Complete System Architecture & Design

Bulk Certificate Generator Backend API

A practical, copy-ready collection of prompts for generating consistent architecture, database, sequence, deployment, data-flow, and workflow diagrams in Eraser.io.

Project baseline	FastAPI · SQLAlchemy · SQLite · ReportLab · Pytest
Processing model	Database-backed Python worker (recommended baseline)
Diagram prompts	12 individually named prompts
Cost objective	Zero-cost local setup; cloud free tiers must be verified

How to use this guide

1. Open Eraser.io and create a new diagram or document.
2. Copy one prompt at a time into its AI diagram-generation feature.
3. Save each result using the suggested filename in this guide.
4. Keep component names, API paths, status values, and database fields consistent across all diagrams.
5. Review each generated diagram for correctness before including it in your report.

Recommended order: High-Level Architecture → ERD → Request Sequence → Deployment Architecture → remaining diagrams.

Diagram Checklist

No.	Diagram	Suggested filename
01	01. High-Level System Architecture	01_High_Level_System_Architecture
02	02. Component Architecture	02_Component_Architecture
03	03. Request Processing Sequence Diagram	03_Request_Processing_Sequence
04	04. Certificate Generation Sequence Diagram	04_Certificate_Generation_Sequence
05	05. Entity Relationship Diagram (ERD)	05_Database_ERD
06	06. Detailed Database Schema Diagram	06_Detailed_Database_Schema
07	07. Deployment Architecture	07_Deployment_Architecture
08	08. Error Handling and Retry Flowchart	08_Error_Handling_Retry_Flow
09	09. Background Worker and Job Lifecycle	09_Background_Worker_Flow
10	10. DFD Level 0 — Context Diagram	10_DFD_Level_0_Context
11	11. DFD Level 1	11_DFD_Level_1
12	12. End-to-End Activity Diagram	12_End_to_End_Activity_Diagram

Consistency rules

• Use FastAPI, SQLAlchemy, SQLite (local), ReportLab, and Pytest consistently unless the implementation deliberately changes.

• Treat the database as persistent job state, not as an execution engine. A Python worker claims and processes jobs.

• A failed recipient should not unnecessarily stop other recipients.

• Distinguish local zero-cost operation from cloud free tiers, which can have changing limits and ephemeral storage.

• Use COMPLETED_WITH_ERRORS when processing finishes with both successful and failed recipients.

01. High-Level System Architecture

Suggested filename: 01_High_Level_System_Architecture

Purpose: Show the complete system from client request to PDF delivery.

COPY-READY ERASER PROMPT

```
Create a professional, technically accurate High-Level System Architecture diagram for a Bulk Certificate Generator Backen

PROJECT CONTEXT:
The system accepts a bulk list of recipients, validates the data, generates individual PDF certificates using one predefin

TECHNOLOGY STACK:
- Client: API consumer using Postman, curl, or another HTTP client
- Backend: Python with FastAPI
- Validation: Pydantic
- Business logic: Certificate Job Service
- PDF generation: ReportLab
- Database access: SQLAlchemy
- Relational database: SQLite for local development
- Background processing: Separate Python worker process that reads persisted jobs from the database
- File storage: Local generated_certificates directory
- API documentation: OpenAPI and Swagger UI
- Testing: Pytest

DIAGRAM COMPONENTS:
1. API Client
2. FastAPI Application
3. API Routes and Request Validation
4. Certificate Job Service
5. Database Layer using SQLAlchemy
6. Relational Database containing GenerationJob and CertificateRecord data
7. Persistent Job Queue represented by pending jobs in the database
8. Background Worker Process
9. Certificate Generation Service
10. Predefined Certificate Template
11. PDF File Storage
12. Job Status and Progress API
13. Certificate Listing and Download API
14. Logging and Error Handling

DATA FLOW:
- The client submits one bulk certificate-generation request.
- FastAPI validates the request and creates a generation job and individual recipient records.
- The database stores the job and recipient statuses.
- The worker claims pending jobs and processes recipients individually.
- The Certificate Generation Service uses the predefined template to create PDF files.
- Successful file references and individual outcomes are persisted in the database.
- The client polls the job-status endpoint to retrieve progress.
- The client retrieves certificate records and downloads successfully generated PDFs through the API.

LAYOUT:
Use a left-to-right architecture with clear boundaries between the client, API layer, business logic, background worker, d

Use solid arrows for synchronous HTTP calls, dashed arrows for background processing or polling where appropriate, and lab

Include a small legend, a clear title, and concise component labels. Avoid decorative icons, excessive detail, and unneces

Do not include Redis, Celery, Kubernetes, paid cloud services, microservices, or an external AI API. Do not imply that the

Output a clean, editable, presentation-ready architecture diagram.
```

02. Component Architecture

Suggested filename: 02_Component_Architecture

Purpose: Show internal FastAPI modules and their responsibilities.

COPY-READY ERASER PROMPT

```
Create a detailed Component Architecture diagram for a modular Python FastAPI Bulk Certificate Generator application.

ARCHITECTURAL STYLE:
Use a layered modular monolith with a separate background worker process. Do not use microservices.

COMPONENTS AND RESPONSIBILITIES:

1. API Layer:
- FastAPI Application Entry Point
- Job Routes
- Certificate Routes
- Health Check Route
- Pydantic Request and Response Schemas

2. Application and Business Logic Layer:
- Certificate Job Service
- Recipient Validation Service
- Job Status and Progress Service
- Certificate Generation Service
- Certificate Retrieval Service

3. Data Access Layer:
- Job Repository
- Certificate Repository
- SQLAlchemy Session Management

4. Persistence Layer:
- GenerationJob Model
- CertificateRecord Model
- Relational Database using SQLite locally

5. Background Processing Layer:
- Job Polling or Claiming Mechanism
- Job Processor
- Recipient Processing Loop
- Job Completion and Failure Handler

6. PDF Generation Layer:
- ReportLab PDF Generator
- Single Predefined Certificate Template
- Unique Certificate ID Generator
- Safe Filename Generator

7. File Storage Layer:
- Generated PDF Directory
- Certificate File Reference Storage
- File Existence and Access Validation

8. Cross-Cutting Components:
- Configuration Management
- Structured Logging
- Exception Handling
- Security and Authorization
- Automated Testing

RELATIONSHIPS:
- Routes call application services.
- Services use repositories to access persistent data.
- Repositories use SQLAlchemy sessions.
- The Job Service creates a job and recipient records.
- The worker claims persisted jobs and invokes the Job Processor.
- The Job Processor invokes the Certificate Generation Service for each recipient.
- The PDF Generator uses the predefined template and saves output files.
- The Certificate Repository stores each recipient's status and certificate file reference.
- The Status Service calculates progress using persisted job and recipient data.
- The Retrieval Service checks authorization and database metadata before serving a PDF.

LAYOUT:
Arrange the diagram in horizontal layers from API to business logic, data access, and persistence. Place the worker and PD

Label the dependencies with action verbs. Use distinct visual groupings for application modules, external interfaces, data

Do not draw direct database access from the API client. Do not make PDF generation part of the database. Do not introduce

Produce a detailed, readable component diagram suitable for a final-year project report and technical interview.
```

03. Request Processing Sequence Diagram

Suggested filename: 03_Request_Processing_Sequence

Purpose: Show how a bulk request is accepted and tracked.

COPY-READY ERASER PROMPT

```
Create a UML Sequence Diagram for submitting a bulk certificate generation request to a FastAPI backend.

PARTICIPANTS FROM LEFT TO RIGHT:
1. API Client
2. FastAPI Job Route
3. Pydantic Validator
4. Certificate Job Service
5. SQLAlchemy Repository
6. Relational Database
7. Background Worker
8. Certificate Generation Service
9. PDF File Storage

MAIN SUCCESS FLOW:
1. API Client sends POST /api/v1/jobs with certificate metadata and a list of recipients.
2. FastAPI Job Route receives the request.
3. Pydantic Validator checks the request structure and field constraints.
4. If the request schema is valid, the Job Service creates a unique job ID.
5. The Job Service creates a GenerationJob record and individual CertificateRecord entries for the recipients.
6. The Repository persists the records in a database transaction.
7. FastAPI returns HTTP 202 Accepted with job_id, status, total_recipients, and a status_url.
8. The Background Worker claims the pending job.
9. The Worker processes each recipient and calls the Certificate Generation Service.
10. Generated PDF files are saved in file storage.
11. Individual recipient outcomes are written to the database.
12. The worker updates the aggregate job status.
13. The client sends GET /api/v1/jobs/{job_id} to check progress.
14. The API retrieves persisted status and counts.
15. The API returns job status, progress, successful count, failed count, and total count.

ALTERNATIVE FLOWS:
- Invalid request schema: return HTTP 422 and do not create a job.
- Empty or business-invalid recipient list: follow explicitly defined validation rules and return a documented client erro
- Database creation failure: roll back the transaction and return a safe server error.
- Individual PDF failure: record the recipient as failed and continue with other recipients.
- Unknown job ID: return HTTP 404.
- Job with mixed recipient outcomes: use COMPLETED_WITH_ERRORS when processing has finished and at least one recipient fai

Use UML lifelines, activation bars, numbered messages, return arrows, and alt/opt frames for conditional paths.

Make it clear that HTTP 202 means the request was accepted for processing, not that every PDF has already been generated.

Do not depict the client waiting for all PDFs to finish. The client should be able to poll job status independently.

Output a precise, readable sequence diagram with labeled request and response messages.
```

04. Certificate Generation Sequence Diagram

Suggested filename: 04_Certificate_Generation_Sequence

Purpose: Detail how an individual recipient's certificate is generated and recorded.

COPY-READY ERASER PROMPT

```
Create a UML Sequence Diagram showing the generation of one recipient's PDF certificate inside a bulk certificate job.

PARTICIPANTS:
1. Background Worker
2. Job Processor
3. Certificate Repository
4. Recipient Validation Logic
5. Certificate Generation Service
6. Certificate ID Generator
7. Predefined Certificate Template
8. ReportLab PDF Renderer
9. File Storage
10. Relational Database

SUCCESS FLOW:
1. Worker claims a pending job.
2. Job Processor loads the recipient's persisted record.
3. Recipient Validation Logic checks required recipient fields and certificate data.
4. Certificate ID Generator creates or retrieves a stable unique certificate identifier.
5. Certificate Generation Service prepares the certificate data.
6. The predefined template receives recipient name, course or event name, completion date, issuing organization, and certi
7. ReportLab renders the certificate as a PDF.
8. File Storage writes the PDF to a safe, unique filename.
9. The service verifies that the output exists and can be opened as a valid PDF.
10. Certificate Repository stores the certificate file reference, identifier, and SUCCESS status.
11. Job Processor updates processing counts and moves to the next recipient.
12. When all recipients have been processed, the worker finalizes the overall job status.

FAILURE FLOW:
- Invalid recipient data: record a validation error for that recipient.
- PDF rendering exception: record a generation error.
- File write failure: do not mark the certificate as successful.
- Database update failure: do not falsely report successful persistence; define recovery behavior for an already-created f
- Any individual recipient failure must not unnecessarily stop other recipients.

Use alt fragments to separate success and failure flows and a loop fragment to represent processing each recipient.

IMPORTANT DESIGN DETAILS:
- Avoid unsafe user-controlled file paths.
- Never overwrite another recipient's certificate.
- Persist status and file references.
- Do not store PDF binary content directly in the database unless there is a specific reason.
- Explain that file creation and database updates are separate operations and require careful error handling.

Use clean UML notation and concise message labels. Output an implementation-oriented sequence diagram rather than a generi
```

05. Entity Relationship Diagram (ERD)

Suggested filename: 05_Database_ERD

Purpose: Define the core database entities, keys, and relationships.

COPY-READY ERASER PROMPT

```
Create a professional Entity Relationship Diagram (ERD) for a Bulk Certificate Generator Backend API.

DATABASE:
Use a relational database with SQLAlchemy. The initial local implementation uses SQLite.

ENTITIES:

1. GenerationJob
- id: UUID or string primary key
- status: string or enum
- total_recipients: integer
- successful_count: integer
- failed_count: integer
- created_at: datetime
- started_at: nullable datetime
- completed_at: nullable datetime
- updated_at: datetime

2. CertificateRecord
- id: UUID or string primary key
- job_id: foreign key referencing GenerationJob.id
- recipient_name: string
- recipient_email: nullable string, if included in the input contract
- certificate_id: unique identifier, nullable until assigned if appropriate
- status: string or enum
- file_path: nullable string
- error_code: nullable string
- error_message: nullable string
- created_at: datetime
- completed_at: nullable datetime

RELATIONSHIPS:
- One GenerationJob has zero or many CertificateRecord records.
- Each CertificateRecord belongs to exactly one GenerationJob.
- Use GenerationJob.id as the primary key of the parent entity.
- Use CertificateRecord.job_id as a foreign key.
- Define appropriate unique constraints and indexes.
- Ensure that the certificate identifier is unique when assigned.
- Explain whether recipient email uniqueness is required within a job or across the entire system; do not impose global un

DESIGN RULES:
- Every submitted recipient must have a persisted outcome record if the job is accepted for processing.
- Store individual success or failure status for each recipient.
- Store a file reference, not the PDF binary, in CertificateRecord.
- Use consistent status values and timestamps.
- Show primary keys, foreign keys, cardinalities, and optional attributes.
- Mark nullable fields appropriately.
- Avoid adding unrelated entities such as payment, subscription, organization, or user management tables unless explicitly

LAYOUT:
Place GenerationJob on the left and CertificateRecord on the right. Show a clear one-to-many relationship with crow's-foot

Include a small legend for PK, FK, unique constraints, and nullable attributes.

Output a database-focused ERD suitable for inclusion in a software engineering project report.
```

06. Detailed Database Schema Diagram

Suggested filename: 06_Detailed_Database_Schema

Purpose: Turn the ERD into a detailed implementation-oriented schema.

COPY-READY ERASER PROMPT

```
Generate a detailed relational Database Schema Diagram for the Bulk Certificate Generator backend.

Use two core tables: generation_jobs and certificate_records.

TABLE 1: generation_jobs
- id: VARCHAR(36), primary key
- status: VARCHAR(32), not null
- total_recipients: INTEGER, not null, default 0
- successful_count: INTEGER, not null, default 0
- failed_count: INTEGER, not null, default 0
- created_at: DATETIME, not null
- started_at: DATETIME, nullable
- completed_at: DATETIME, nullable
- updated_at: DATETIME, not null

TABLE 2: certificate_records
- id: VARCHAR(36), primary key
- job_id: VARCHAR(36), not null, foreign key to generation_jobs.id
- recipient_name: VARCHAR(255), not null
- recipient_email: VARCHAR(320), nullable if email is optional
- certificate_id: VARCHAR(100), nullable until assigned, unique when present
- status: VARCHAR(32), not null
- file_path: TEXT, nullable
- error_code: VARCHAR(64), nullable
- error_message: TEXT, nullable
- created_at: DATETIME, not null
- completed_at: DATETIME, nullable

RELATIONSHIPS AND CONSTRAINTS:
- generation_jobs.id → certificate_records.job_id
- One-to-many relationship.
- Index certificate_records.job_id.
- Consider an index on generation_jobs.status for worker polling.
- Enforce allowed status values in application validation or database constraints.
- Enforce unique certificate IDs when populated.
- Use appropriate foreign-key behavior.
- Define timestamps consistently in UTC.
- Ensure counter updates remain consistent with individual recipient statuses.

WORKER SUPPORT:
If background processing is used, show how pending jobs are identified and claimed. If concurrency is possible, include a

OUTPUT:
Draw tables with column names, data types, PK/FK markers, nullability, and constraints. Show indexes separately if support

Add a note that the schema is a proposed implementation design and that exact types and constraints must be checked agains

Do not add Redis, external queues, or unrelated business tables. Create a clean, technically accurate schema diagram.
```

07. Deployment Architecture

Suggested filename: 07_Deployment_Architecture

Purpose: Show local deployment and a conditional cloud deployment option.

COPY-READY ERASER PROMPT

```
Create a Deployment Architecture diagram for a zero-cost Bulk Certificate Generator Backend API.

SHOW TWO SEPARATE DEPLOYMENT OPTIONS.

OPTION A: LOCAL DEVELOPMENT AND DEMONSTRATION
- Developer workstation
- Python virtual environment
- FastAPI application running through Uvicorn
- Background worker as a separate Python process
- SQLite database stored on local disk
- Persistent generated_certificates directory
- Environment configuration from .env
- Swagger UI for API testing
- Git repository for source control
- Pytest test runner

Show that the API and worker access the same persisted database and shared file-storage directory.

OPTION B: OPTIONAL CLOUD DEPLOYMENT
- API client accessing the service over HTTPS
- Cloud-hosted Python web application
- Separate worker process if supported by the chosen platform
- Managed relational database or another supported persistent database
- Persistent object or file storage
- Environment-based secrets and configuration
- Application logs and health checks

IMPORTANT CLOUD RULES:
- Do not select a provider solely because it advertises free hosting.
- Verify current free-tier pricing, database availability, worker support, sleep behavior, storage persistence, resource l
- Label cloud components as conditional on provider support and cost verification.
- Do not imply that local SQLite files or local generated PDF files automatically persist in cloud environments.
- Do not show a single ephemeral application filesystem as reliable persistent storage.
- If no suitable zero-cost cloud setup meets the requirements, show local deployment as the fully functional fallback.

SECURITY:
- Show HTTPS at the cloud boundary.
- Show environment-based secrets.
- Restrict direct access to database and file storage.
- Include authentication and authorization at the API layer when certificates are not intended to be publicly accessible.

LAYOUT:
Use separate containers for local deployment and cloud deployment. Show process boundaries, network access, persistent res

Use a professional deployment diagram with clear labels and a note distinguishing free local operation from potentially li
```

08. Error Handling and Retry Flowchart

Suggested filename: 08_Error_Handling_Retry_Flow

Purpose: Document validation errors, per-recipient failures, and bounded retries.

COPY-READY ERASER PROMPT

```
Create a detailed software error-handling flowchart for a Bulk Certificate Generator backend.

START:
Client submits a bulk certificate generation request.

REQUEST VALIDATION:
- Validate JSON structure and required fields.
- If schema validation fails, return HTTP 422.
- Check the configured recipient batch limit.
- If the limit is exceeded, return a documented client error.
- Apply the defined business rules to recipient data.
- If the request is valid, create a GenerationJob and its recipient records.
- Persist the accepted job and return HTTP 202 with the job ID.

JOB PROCESSING:
- Worker claims a pending job.
- Worker processes each recipient independently.
- Validate recipient-specific fields.
- If recipient validation fails, record a validation error and continue.
- Otherwise, generate the PDF using the predefined template.
- If PDF generation succeeds, save the file and persist its reference and success status.
- If PDF generation fails, record the failure and continue to the next recipient.
- If file storage fails, do not mark the certificate as successful.
- If a database update fails, log the issue and apply the documented recovery strategy.

RETRY POLICY:
- Distinguish permanent errors, such as invalid input, from transient errors, such as temporary storage or database failur
- Retry only explicitly classified transient failures.
- Limit retries and use backoff where appropriate.
- Avoid generating duplicate certificates during retries.
- Use a stable certificate ID or idempotency mechanism when justified.
- If retries are exhausted, record a terminal failure or flag the job for recovery.
- Do not automatically retry every error indefinitely.

JOB FINALIZATION:
- Determine whether all recipient records have reached terminal states.
- If all succeed, mark the job COMPLETED.
- If processing finishes with a mixture of successes and failures, mark it COMPLETED_WITH_ERRORS.
- If the entire job cannot be processed, mark it FAILED only according to the defined job-level failure policy.
- Update timestamps and aggregate counters.
- Make the final status available through the job-status endpoint.

Use decision diamonds for validation and generation outcomes, process boxes for database updates, and clearly labeled succ

Include a visible distinction between recipient-level failures and job-level failures. An individual certificate failure m

Produce a clean, logically complete flowchart with no circular retry loop lacking a maximum retry limit.
```

09. Background Worker and Job Lifecycle

Suggested filename: 09_Background_Worker_Flow

Purpose: Explain asynchronous job processing, persistence, and recovery.

COPY-READY ERASER PROMPT

```
Create a Background Worker Architecture and Job Lifecycle Flowchart for a zero-cost FastAPI Bulk Certificate Generator.

PROCESSING MODEL:
Use a separate Python worker process that retrieves persisted pending jobs from a relational database. Do not require Redi

JOB LIFECYCLE:
1. Client submits a bulk generation request.
2. API validates the request.
3. API creates a GenerationJob record with PENDING status and individual CertificateRecord entries.
4. API commits the accepted job to the database.
5. API returns HTTP 202 with the job ID.
6. Worker polls for pending jobs.
7. Worker claims a job and transitions it to PROCESSING using a safe claim strategy.
8. Worker processes recipient records one at a time or in controlled batches.
9. Each recipient transitions through appropriate states, such as PENDING, PROCESSING, SUCCESS, or FAILED.
10. Worker generates PDFs and saves successful outputs.
11. Worker persists individual results and updates job progress.
12. Worker continues until all recipients are processed or a job-level interruption occurs.
13. Worker determines the final aggregate job status.
14. Client polls the status endpoint and retrieves completed certificates.

RELIABILITY:
- Explain atomic claiming or a single-worker limitation.
- Avoid two workers processing the same job simultaneously.
- Persist state so it survives process restarts.
- Define behavior for interrupted jobs and stale PROCESSING states.
- Consider leases, timestamps, bounded retries, and recovery policies where appropriate.
- Avoid assuming that SQLite supports the same concurrency and locking behavior as a production database.
- Avoid holding a database transaction open during lengthy PDF generation.
- Ensure individual failures do not stop unrelated recipient processing.
- Keep job counters consistent with recipient records.

STATUS MODEL:
PENDING → PROCESSING → COMPLETED
PENDING → PROCESSING → COMPLETED_WITH_ERRORS
PROCESSING → FAILED only for a defined job-level failure
Interrupted jobs require an explicit recovery policy before returning to PROCESSING.

LAYOUT:
Use a lifecycle flowchart with decision points for pending jobs, recipient processing, individual failure, retry eligibili

Show the database as the source of persisted job state and the worker as the execution process. Show the API as the client

Include a note that database-backed polling is a simple zero-cost baseline, not automatically a high-throughput distribute

Produce a professional, implementation-ready diagram.
```

10. DFD Level 0 — Context Diagram

Suggested filename: 10_DFD_Level_0_Context

Purpose: Show the system boundary and external data flows.

COPY-READY ERASER PROMPT

```
Create a Data Flow Diagram (DFD) Level 0, also known as a Context Diagram, for the Bulk Certificate Generator Backend Syst

Represent the entire application as one process:
Process 0: Bulk Certificate Generator System.

EXTERNAL ENTITY:
- API Client: submits recipient information and certificate metadata, requests job progress, lists certificates, and downl

INPUT DATA FLOWS:
- Bulk generation request
- Recipient details
- Certificate metadata
- Job status request
- Certificate retrieval request

OUTPUT DATA FLOWS:
- Request validation response
- Job ID and accepted status
- Job progress and processing results
- Certificate list and individual status
- Generated PDF download
- Appropriate error responses

SYSTEM RESPONSIBILITIES:
- Validate requests
- Generate certificates in bulk
- Track job and recipient outcomes
- Store PDF files and metadata
- Return progress and retrieval results

DFD RULES:
- Use one main process.
- Show the API Client as an external entity.
- Show labeled arrows for input and output data flows.
- Do not show internal Python modules or individual database tables.
- Keep the system boundary clear.
- Do not confuse a data flow with a control-flow arrow.

Output a clean, standard DFD Level 0 suitable for a software engineering project report.
```

11. DFD Level 1

Suggested filename: 11_DFD_Level_1

Purpose: Decompose the system into processes and data stores.

COPY-READY ERASER PROMPT

```
Create a DFD Level 1 for the Bulk Certificate Generator Backend, decomposing the main system into the following numbered p

EXTERNAL ENTITY:
- E1: API Client

PROCESSES:
- 1.0: Receive and Validate Bulk Request
- 2.0: Create and Track Generation Job
- 3.0: Process Recipients and Generate Certificates
- 4.0: Update Job and Certificate Status
- 5.0: Retrieve Job Status and Certificate Files

DATA STORES:
- D1: Generation Jobs
- D2: Certificate Records
- D3: Generated PDF Files

DATA FLOWS:
- E1 sends the bulk request to 1.0.
- 1.0 validates the request and forwards valid data to 2.0.
- 2.0 creates the job and recipient records, writing job information to D1 and recipient information to D2.
- 3.0 reads pending jobs and recipients, generates PDFs, and stores files in D3.
- 4.0 updates individual outcomes in D2 and aggregate job status in D1.
- E1 requests status from 5.0, which reads D1 and D2.
- E1 requests certificates from 5.0, which checks D2 and retrieves authorized files from D3.

VALIDATION AND FAILURE:
Show invalid requests returning an error to E1. Show recipient-level generation failures recorded in D2 without stopping t

DFD RULES:
- Use numbered processes, external entities, data stores, and labeled data flows.
- Do not connect an external entity directly to a data store.
- Keep all flows balanced with the context diagram.
- Use data-flow labels such as recipient data, job metadata, certificate file, and processing result.
- Do not use UML lifelines or control-flow decision diamonds.

Output a readable DFD Level 1 with consistent notation and clearly separated processes and data stores.
```

12. End-to-End Activity Diagram

Suggested filename: 12_End_to_End_Activity_Diagram

Purpose: Show the complete workflow from request submission to retrieval.

COPY-READY ERASER PROMPT

```
Create a UML Activity Diagram for the complete Bulk Certificate Generator application workflow.

START:
Client submits a bulk certificate generation request.

ACTIVITIES:
1. Receive request.
2. Validate request schema and batch size.
3. Create generation job and recipient records.
4. Return the job ID and accepted status.
5. Worker claims the job.
6. Retrieve the next recipient.
7. Validate recipient-specific information.
8. Generate the PDF using the predefined certificate template.
9. Save the PDF and persist its reference.
10. Record recipient success or failure.
11. Update job progress.
12. Check whether additional recipients remain.
13. Finalize the aggregate job status.
14. Allow the client to retrieve status and generated certificates.

DECISIONS:
- Is the request valid?
- Are recipient fields valid?
- Was the certificate generated successfully?
- Were all recipients processed?
- Is the requested certificate available and authorized for retrieval?

ALTERNATE PATHS:
- Invalid request: return an error and terminate the request flow.
- Invalid recipient: record the error and continue.
- PDF generation failure: record the failure and continue.
- All recipients processed: set COMPLETED or COMPLETED_WITH_ERRORS based on outcomes.
- Unknown job or certificate: return HTTP 404.
- Unauthorized download: reject access according to the security policy.

Use UML activity symbols, decision diamonds, start/end nodes, and labeled branches. Separate client-facing actions from ba

Keep the workflow consistent with a database-backed asynchronous implementation and avoid suggesting that a client must wa

Output a clear end-to-end activity diagram suitable for project documentation.
```

Final Review Before Submission

- ■ All diagrams describe the same implementation and technology stack.
- ■ The ERD and database schema agree with the actual SQLAlchemy models.
- ■ The API paths and HTTP status codes match the implemented endpoints.
- ■ Sequence and activity diagrams distinguish request acceptance from background completion.
- ■ The worker design explains job claiming, recovery, and individual failure isolation.
- ■ The deployment diagram distinguishes persistent storage from ephemeral storage.
- ■ Cloud providers and free-tier limits are verified from current official documentation before use.
- ■ Every diagram is reviewed and corrected after Eraser.io generates it.

Note: These prompts describe a proposed design. Ensure the final diagrams reflect the code and architecture actually implemented.